

Lecture 1: Introduction to Discrete Structures

Course: Discrete Structures (MATH-121) | Unit 1: Foundations

1. What Is Discrete Structures?

Discrete Structures is the branch of mathematics that studies objects which are **countable, separate, and distinct** rather than continuous. Where calculus deals with smooth, unbroken change — velocities, curves, areas — discrete mathematics deals with things you can count on your fingers: statements that are true or false, finite sets of items, relationships between objects, and step-by-step procedures. This makes it, in a very real sense, **the mathematics of computer science**: a computer is fundamentally a discrete machine, operating on bits (0 or 1), finite memory, and step-by-step instructions.

This course, MATH-121, is designed to build the logical and mathematical toolkit that every computer scientist, software engineer, and data scientist relies on — and to show, at every step, how that toolkit turns directly into working Python code.

2. What We Will Study

Over the semester, this course is organized into four broad units, moving from the language of precise reasoning to the structures that describe data and computation:

Unit 1 – Logic & Proof

Propositional logic, logical connectors, truth tables, arguments and rules of inference — the language of precise, unambiguous reasoning.

Unit 2 – Sets, Relations & Functions

Set theory, relations (properties, equivalence, partial orders), and functions (types, inverses, composition) — the structures used to model data and mappings between data.

Unit 3 – Sequences & Series

Arithmetic and geometric sequences and series, recurrence relations, and summation — the backbone of algorithm analysis and iterative processes.

Unit 4 – Combinatorics

Counting principles, permutations and combinations — the mathematics of "how many ways," essential for probability, complexity analysis, and cryptography.

3. Why Study Discrete Structures?

It is fair to ask: why does a Computer Science student need a mathematics course at all? The honest answer is that discrete mathematics is not *alongside* computer science — it is largely *underneath* it. Consider a few concrete reasons:

- **Programming logic:** every `if`, `while`, and `and/or/not` in your code is propositional logic in disguise.
- **Data structures:** sets, relations, and functions are the mathematical blueprints for Python's `set`, `dict`, and graph-based structures.
- **Algorithm analysis:** sequences, series, and recurrence relations are exactly how we predict how long an algorithm will take as input grows (Big-O analysis).
- **Databases:** relations in the mathematical sense are literally where the word "relational database" comes from.
- **AI & Machine Learning:** combinatorics and counting underlie probability, which is the foundation of every ML model's confidence and uncertainty.
- **Cryptography & security:** number theory, sets, and combinatorics protect every password and every HTTPS connection you use.

4. Connecting Theory to Real-World Computer Science

A distinctive feature of this course is that every topic will be paired with a small, runnable **Python** example. Python is an ideal companion language here: its syntax for sets, logical operators, and functions maps almost one-to-one onto the mathematical notation we will use on the board. The table below previews how each unit of mathematics shows up directly in code and in real systems you already use.

Mathematical Topic	Python Feature	Real-World Application
Logical connectors ($\wedge, \vee, \neg$)	<code>and</code> , <code>or</code> , <code>not</code>	Search filters, login checks, if-conditions
Sets	<code>set()</code> , <code>union</code> , <code>intersection</code>	Removing duplicates, recommendation engines
Relations / Functions	<code>dict</code> , <code>def f(x):</code>	APIs, databases, hash maps
Sequences / Series	<code>for</code> loops, <code>sum()</code>	Loop invariants, algorithm running time
Combinatorics	<code>itertools.permutations</code>	Password strength, scheduling, ML feature combos

A First Taste: Logic in Python

Here is a tiny preview of what is coming in Lecture 2. A logical statement such as "a student passes if their attendance is at least 75% **and** their marks are at least 50" translates directly into Python:

```
attendance = 80    # percent
marks = 55         # percent

passes = (attendance >= 75) and (marks >= 50)
print("Result:", passes)    # Result: True
```

Notice that the English word "and" became Python's **and** keyword, which is exactly the logical connector $\wedge$ (conjunction) that we will formally define in the next lecture.

5. How This Course Will Run

Structure of Every Lecture

Each lecture in this series follows the same pattern: (1) core mathematical **definitions**, (2) **worked examples**, (3) a short **Python implementation** of the idea, and (4) **practice questions** for you to attempt before the next class.

By the end of this course, you will be able to read and write formal logical statements, reason about sets and relations with confidence, analyze the growth of sequences and algorithms, count complex arrangements systematically, and — throughout — express all of these ideas as working Python code. This is not mathematics for its own sake; it is the shared language that lets you talk precisely to a computer, to a database, and to a proof.

Next lecture: Propositional Logic — statements, logical connectors, and their implementation in Python.